

TESI FINALE DI PROGETTAZIONE DI SISTEMI OPERATIVI

## Programmazione concorrente nel Linguaggio Java

Approfondimento sull'utilizzo dei *Thread* in Java

Studente:

**Tommaso Cosimi**

Matricola 191739

Docente:

**Prof.ssa Letizia Leonardi**

Codocente:

**PhD Silvia Cascianelli**



## Sommario

Nel corso della Storia dell'Informatica, l'avanzamento tecnico e tecnologico verificatosi nell'ambito delle Architetture dei Calcolatori e dei Sistemi Operativi ha permesso grandi passi in avanti nelle dinamiche di utilizzo degli Elaboratori Elettronici. Si è assistito al passaggio dall'esecuzione sequenziale delle operazioni ad un'esecuzione parallela delle stesse, il tutto all'interno di un singolo Elaboratore.

L'opportunità di avere esperienze utente multiprogrammate è stata resa possibile, oltre che dagli Elaboratori non sequenziali, anche da Linguaggi di Programmazione anch'essi non sequenziali che permettano l'esecuzione concorrente di più processi. Tale flessibilità ha permesso un grado di utilizzo delle risorse nettamente superiore al passato, garantendo prestazioni superiori a parità di risorse.

Lo scopo di questo elaborato sarà quello di approfondire l'argomento della Programmazione Concorrente nel Linguaggio Java, riportando allo stesso tempo esempi pratici di utilizzo.



# Indice

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                                       | <b>11</b> |
| 1.1      | Algoritmo, Programma, Processo . . . . .                                  | 11        |
| 1.2      | Stati di un Processo . . . . .                                            | 13        |
| 1.3      | Esecuzione Sequenziale e non Sequenziale . . . . .                        | 15        |
| 1.3.1    | Processi Sequenziali . . . . .                                            | 15        |
| 1.3.2    | Processi non Sequenziali . . . . .                                        | 16        |
| 1.4      | Processi Interagenti . . . . .                                            | 17        |
| 1.4.1    | Interazione Indiretta (Competizione) . . . . .                            | 17        |
| 1.4.2    | Interazione Diretta (Cooperazione) . . . . .                              | 18        |
| 1.4.3    | Interazione Indesiderata (Interferenza) . . . . .                         | 18        |
| 1.5      | Sincronizzazione tra Processi . . . . .                                   | 18        |
| 1.5.1    | Sezione Critica . . . . .                                                 | 19        |
| 1.5.2    | Strumenti di Sincronizzazione . . . . .                                   | 19        |
| 1.5.3    | Modelli Logici per la Concorrenza . . . . .                               | 25        |
| 1.6      | Processi Pesanti e Processi Leggeri . . . . .                             | 25        |
| 1.6.1    | Processi Pesanti . . . . .                                                | 25        |
| 1.6.2    | Processi Leggeri - “ <i>Thread</i> ” . . . . .                            | 26        |
| <b>2</b> | <b><i>Multi-Threading</i> in Java</b>                                     | <b>29</b> |
| 2.1      | La Classe “ <b>Thread</b> ” e l’Interfaccia “ <b>Runnable</b> ” . . . . . | 29        |
| 2.2      | Stesura di un semplice Programma Multiprocesso . . . . .                  | 30        |
| 2.2.1    | Implementazione come Sotto-Classe di <b>Thread</b> . . . . .              | 30        |
| 2.2.2    | Implementazione dell’Interfaccia <b>Runnable</b> . . . . .                | 31        |
| 2.3      | Controllo del Flusso di Esecuzione . . . . .                              | 33        |
| 2.3.1    | Problemi del Controllo del Flusso di Esecuzione . . . . .                 | 35        |
| 2.3.2    | Considerazioni sul Controllo del Flusso di Esecuzione . . . . .           | 36        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>3</b> | <b>Sincronizzazione di Processi Leggeri in Java</b> | <b>37</b> |
| 3.1      | Il Costrutto <i>Synchronized</i> . . . . .          | 37        |
| 3.2      | Il costrutto <i>Lock</i> . . . . .                  | 38        |

# Elenco delle figure

|     |                                                                       |    |
|-----|-----------------------------------------------------------------------|----|
| 1.1 | Transizioni tra gli Stati di un Processo . . . . .                    | 14 |
| 1.2 | Transizioni tra gli Stati di un Processo in UNIX . . . . .            | 15 |
| 1.3 | Grafo delle Precedenze di un Processo Sequenziale . . . . .           | 16 |
| 1.4 | Grafo delle Precedenze di un Processo non Sequenziale . . . . .       | 17 |
| 1.5 | Esempio di Interazione Competitiva tra due Processi P1 e P2 . . . . . | 17 |
| 2.1 | Ereditarietà della Classe <b>Thread</b> . . . . .                     | 29 |
| 2.2 | Flusso di Esecuzione alterato dal Metodo “ <b>join()</b> ” . . . . .  | 35 |
| 3.1 | Esempio di utilizzo del Costrutto <i>Synchronized</i> . . . . .       | 38 |



## Elenco delle tabelle



# Elenco degli algoritmi

|   |                                              |    |
|---|----------------------------------------------|----|
| 1 | <i>Bubble Sort</i> . . . . .                 | 12 |
| 2 | Esempio di utilizzo di un Semaforo . . . . . | 20 |
| 3 | Pseudocodice di un Semaforo . . . . .        | 21 |
| 4 | Pseudocodice di un Monitor . . . . .         | 22 |



# Capitolo 1

## Introduzione

Il seguente Capitolo fornirà un'introduzione teorica al concetto di esecuzione non sequenziale di un processo, ponendo enfasi sui vantaggi da essa portati e sulle nuove necessità richieste da tale implementazione.

### 1.1 Algoritmo, Programma, Processo

Nella definizione della singola unità di esecuzione si ritiene opportuno effettuare una leggera digressione.

#### Algoritmo

Un Algoritmo è un procedimento puramente logico studiato appositamente per la risoluzione di uno specifico problema. In letteratura sono rintracciabili moltissimi Algoritmi, per brevità vengono nominati a titolo di esempio Algoritmi di Ordinamento come il *Bubble Sort*[1] - di cui viene fornito lo pseudocodice a seguire - od il *Radix Sort*[2], e Algoritmi di Ricerca del Cammino Minimo in un Grafo come l'*Algoritmo di Dijkstra*[3] o l'*Algoritmo di Bellman-Ford*[4].

---

**Algoritmo 1:** Ordinamento per confronti “*Bubble Sort*”

```

1 Data: A; // Vettore di numeri interi non ordinati
2 Result: A; // Vettore di numeri interi ordinati
3 partizioneNonOrdinata  $\leftarrow$  A.length;
4 while partizioneNonOrdinata > 1 do
5     for i = 2 fino a k do
6         if A[i] < A[i - 1] then
7             Scambia A[i] ed A[i - 1];
8         end if
9     end for
10    partizioneNonOrdinata  $\leftarrow$  (ultimoScambio - 1);
11 end while
12 return A

```

# Programma

Un Programma è la descrizione in codice di un Algoritmo in uno specifico Linguaggio di Programmazione in maniera tale da permetterne l'esecuzione all'interno di un Calcolatore. Si riporta di seguito un'esempio di Programma che descriva l'Algoritmo di Ordinamento per confronti “*Bubble Sort*” implementato tramite il Linguaggio di Programmazione Java.

```

1 public class BubbleSort {
2
3     public static void BubbleSort(int[] A) {
4
5         // Definizione della dimensione della partizione del vettore non
6         ↪ ordinata
7
8         int unorderedPartition = A.length;
9
10        // Ordinamento
11        while(unorderedPartition > 1) {
12            // Scorro la parte non ordinata
13            for(int i=2; i<unorderedPartition; i++) {
14                // Se non e' ordinato, scambia
15                if(A[i] < A[i-1]) {
16                    int buf = A[i];
17                    A[i] = A[i-1];
18                    A[i-1] = buf;
19                }
20            }
21        }
22    }
23 }

```

```
18     }
19     // Dopo lo scambio, riduco la dimensione della partizione del
        ↳ vettore non ordinata
20     unorderedPartition = unorderedPartition - 1;
21 }
22
23 }
24
25 }
```

## Processo

Un Processo è la sequenza di operazioni eseguite da un Elaboratore che opera sotto il controllo di un Programma: in maniera intuitiva è possibile affermare che in linea generale un Processo è un Programma in esecuzione. Di seguito si riporta un esempio di esecuzione sequenziale di due Processi che eseguono lo stesso Programma, ovvero *Bubble Sort*.

```
$ java BubbleSort 1 2 3 6 5 2
Il vettore non ordinato e': [1, 2, 3, 6, 5, 2]
Il vettore ordinato e': [1, 2, 2, 3, 5, 6]
$ java BubbleSort 1 9 8 3 6 7
Il vettore non ordinato e': [1, 9, 8, 3, 6, 7]
Il vettore ordinato e': [1, 3, 6, 7, 8, 9]
```

## 1.2 Stati di un Processo

Un Processo può passare attraverso diversi Stati durante il suo ciclo di vita. Nel caso più generale è possibile individuarne essenzialmente tre:

1. **Esecuzione** (o *Running*).

Il Processo detiene il controllo della CPU dell'Elaboratore e sta eseguendo il proprio codice.

2. **Bloccato** (o *Blocked*).

Il Processo non detiene il controllo della CPU dell'Elaboratore e rimane in uno stato dormiente ove attende di essere pronto a richiederne il controllo. Un Processo è in stato *Blocked* tipicamente durante operazioni di I/O mentre è in attesa di dati provenienti da dispositivi meno veloci o dall'Utente.

### 3. Pronto (o *Ready*).

Anche in questo caso il Processo non detiene il controllo della CPU dell'Elaboratore, ma si trova in una coda di Processi che si dicono pronti a richiederne il controllo. Un Processo è in stato *Ready* tipicamente al termine delle suddette operazioni di I/O mentre attende che il controllo della CPU passi dal Processo attualmente in stato *Running* ad esso.

Le transizioni di Stato che possono avvenire dipendono dall'Algoritmo di *Scheduling* del Sistema Operativo, e sono ben descritte nello schema a seguire.

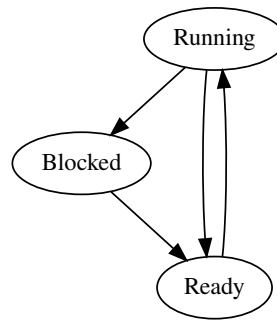


Figura 1.1: Grafo Diretto raffigurante le possibili Transizioni tra i possibili Stati di un Processo. Grafo Generato tramite l'ausilio dell'editor PlantText[5].

Tutte le transizioni di Stato, a meno di quella che porta dallo Stato “*Running*” allo Stato “*Ready*” sono possibili in ogni Algoritmo di Scheduling, mentre quest'ultima è implementabile se e solo se il suddetto Algoritmo supporta meccanismi di Prelazione.

Ovviamente la rappresentazione riportata è un caso esemplificativo utile alla comprensione di alcuni comportamenti rintracciabili nella sincronizzazione tra processi nel Linguaggio Java, ma si ricorda che il Grafo degli Stati e delle loro transizioni in un Sistema Operativo moderno risulta nettamente più complicato, si riporta a seguire quello di UNIX.

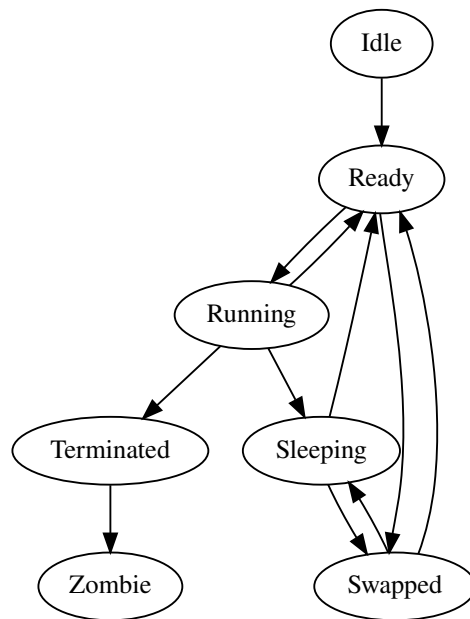


Figura 1.2: Grafo Diretto raffigurante le possibili Transizioni tra i possibili Stati di un Processo in UNIX. Grafo Generato tramite l'ausilio dell'editor PlantText[5].

### 1.3 Esecuzione Sequenziale ed Esecuzione non Sequenziale

La sequenza di operazioni eseguite da un processo può essere rappresentata attraverso un Grafo Orientato che indichi la precedenza delle une sulle altre. In questa visualizzazione i nodi identificano il singolo evento da verificarsi, mentre gli archi l'ordinamento temporale tra i nodi.

#### 1.3.1 Processi Sequenziali

Facendo riferimento alla rappresentazione descritta in precedenza, un Processo Sequenziale è tale da poter essere schematizzato tramite l'ausilio di un Grafo ad Ordinamento Totale: una particolare tipologia di Grafo ove ogni nodo - e quindi ogni evento - fatta eccezione per il nodo di inizio ed il nodo di fine, possiede uno e un solo predecessore ed uno e un solo successore.

A seguire viene effettuato un esempio di Processo Sequenziale.

### Esempio di Rappresentazione di un Processo Sequenziale

Si riporta a titolo di esempio un Processo che richiede all'utente due numeri interi e ne restituisce la somma. Esso è rappresentabile come una sequenza ordinata di operazioni atomiche come esplicitato a seguire.

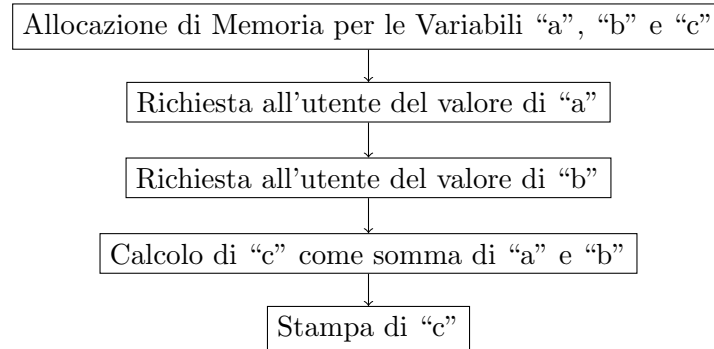


Figura 1.3: Grafo delle Precedenze di un Processo Sequenziale.

### 1.3.2 Processi non Sequenziali

Al contrario della controparte, un Processo non Sequenziale può essere rappresentato tramite l'ausilio di un Grafo ad Ordinamento Parziale: ogni nodo interno potrà quindi avere più predecessori e più successori. Questo è possibile in quanto alcune operazioni eseguite dal Processo sono indipendenti tra loro o non è interessante il loro ordine di terminazione per l'utente finale.

A seguire viene effettuato un esempio di Processo non Sequenziale.

### Esempio di Rappresentazione di un Processo non Sequenziale

Si supponga di dover valutare l'espressione matematica:

$$(5 + 8) + (6 - 3) * (9 + 2)$$

Ovviamente è possibile eseguire sequenzialmente la valutazione verificando prima i risultati delle operazioni tra parentesi, poi la moltiplicazione tra gli ultimi due termini ed infine l'addizione tra i primi due. La natura del problema permette tuttavia di eseguire in ordine arbitrario dei sottoinsiemi di operazioni, i quali risultati risultano indipendenti da un punto di vista

di utilizzo dei dati: è possibile in effetti delegare la valutazione di singoli blocchi a differenti agenti di calcolo.

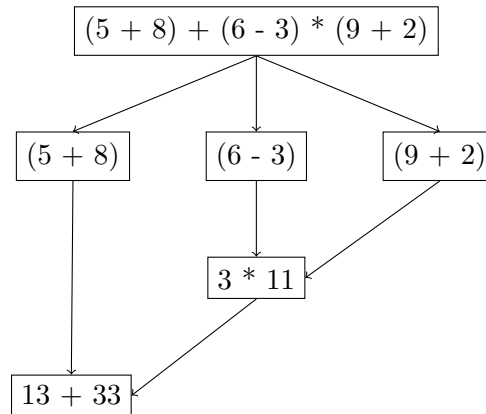


Figura 1.4: Grafo delle Precedenze di un Processo non Sequenziale.

## 1.4 Processi Interagenti

Dall'esecuzione non sequenziale è possibile ottenere situazioni ove due o più Processi debbano interagire tra loro per il completamento delle rispettive operazioni. È possibile individuare tre possibili tipi di interazione.

### 1.4.1 Interazione Indiretta (Competizione)

In questo caso i suddetti Processi competono per l'utilizzo di risorse comuni come variabili o dispositivi di Input/Output. Un esempio lampante potrebbe essere la condivisione di una variabile numerica per due Processi P1 e P2.

```
// Variabile Globale
int x = 10;
```

Esecuzione di P1:

```
...
x = x + 5;
...
```

Esecuzione di P2:

```
...
x = x + 10;
...
```

Figura 1.5: Esempio di Interazione Competitiva tra due Processi P1 e P2. Entrambi i Processi illustrati vanno ad operare sulla Variabile Globale "x".

La modifica contemporanea della Variabile Globale “ $x$ ” da parte di P1 e P2 potrebbe risultare problematica in quanto la lascerebbero in uno stato inconsistente.

Alla base di questa interazione si trova dunque il problema della Mutua Esclusione: una risorsa può essere utilizzata da uno e un solo Processo alla volta. Al fine di una corretta esecuzione bisognerà impossibilitare l'esecuzione contemporanea di Operazioni che facciano utilizzo di risorse comuni.

#### 1.4.2 Interazione Diretta (Cooperazione)

La Cooperazione prevede invece lo scambio delle informazioni da un Processo ad un altro. Si prenda in esame la Figura 1.4: il Processo che andrà a valutare l'espressione finale dovrà sicuramente ottenere i risultati del Processo che ha calcolato la prima parentesi e di quello che ha calcolato la moltiplicazione immediatamente successiva.

Al fine di preservare la correttezza dell'Operazione si renderà necessario far iniziare le Operazioni del Processo finale successivamente al termine delle Operazioni degli altri due Processi.

#### 1.4.3 Interazione Indesiderata (Interferenza)

Quest'ultima è un effetto indesiderato dato da errori di sincronizzazione tra i processi.

L'Interferenza viene risolta tramite l'utilizzo dei costrutti di sincronizzazione atti alla risoluzione dei problemi nati dalla Competizione e dalla Cooperazione citati in precedenza.

### 1.5 Sincronizzazione tra Processi

Alla luce di quanto esposto risulta importante esplicitare quali siano gli strumenti e le tecniche di sincronizzazione utilizzate per garantire la correttezza dei risultati di Processi Concorrenti. Prima di passare ad una loro descrizione è comunque necessario introdurre il concetto di Sezione Critica.

### 1.5.1 Sezione Critica

La Sezione Critica viene definita come una serie di Operazioni nelle quali un Processo accede e/o modifica risorse comuni a più Processi.

Diverse Sezioni Critiche che accedano a gruppi differenti di Risorse possono essere scisse in altrettante categorie.

L'idea alla base dell'esistenza della Sezione Critica è quella di permettere ad uno ed un solo Processo alla volta di entrare in tale segmento di Codice, in maniera tale da garantire un accesso mutualmente esclusivo alle risorse comuni in essa manipolate.

### 1.5.2 Strumenti di Sincronizzazione

Gli Strumenti illustrati di seguito non rappresentano l'interezza delle metodologie di Sincronizzazione disponibili in letteratura, ma sono stati scelti in maniera tale da essere il più possibile utili al fine di introdurre i concetti alla base del *Multi-Threading* nel Linguaggio Java ed inoltre chiarificare alcune metodologie utilizzate anche al di fuori di quest'ultimo (si veda lo Scambio di Messaggi).

Lo scopo di tali Meccanismi sarà quello di permettere al più ad un processo di accedere ad ogni categoria di Sezione Critica. Per ottenere questo risultato, la Sezione Critica va incapsulata da un Prologo che la preceda ed un Epilogo che la segua. In maniera intuitiva si può pensare al Prologo e all'Epilogo come l'operazione di acquisizione di un lucchetto per bloccare una risorsa per poi rilasciarlo al prossimo Processo che ne necessita rispettivamente.

A seconda del tipo di Interazione tra i Processi presi in esame è importante definire la corretta metodologia di Sincronizzazione: per Interazioni di tipo Competitivo si utilizzano tipicamente strumenti quali Semafori e Monitor, mentre per Interazioni di tipo Cooperativo si usufruisce della possibilità di scambio di Messaggi tra Processi. Si riportano a seguire i tre Meccanismi citati.

#### Semaforo

Il Semaforo[6], introdotto dall'informatico olandese Edsger Wybe Dijkstra, è un tipo di dato astratto (ADT) atto alla sincronizzazione che si compone di:

- Valore Intero  $S_v \geq 0$ ;
- Coda di Descrittori  $Q_s$ .

E sul quale è possibile effettuare le operazioni di:

- “wait(S)”;
- “signal(S)”.

Il Semaforo va ad incapsulare la Sezione Critica al fine di rendere possibile ad un solo processo alla volta l’accesso ad essa.

La variabile  $S_v$  mantiene come informazione quanti processi possono accedere alla loro Sezione Critica (se vale zero il processo si sospende sul Semaforo), mentre  $Q_s$  è una Coda di Descrittori dei Processi che non sono riusciti ad accedere alla loro Sezione Critica a causa del Semaforo.

Le Operazioni “wait(S)” e “signal(S)” vanno invece rispettivamente a decrementare ed aumentare il valore della variabile  $S_v$  del Semaforo.

L’utilizzo di un Semaforo per la protezione di una Sezione Critica avviene secondo il seguente schema:

---

**Algoritmo 2:** Esempio di utilizzo di un Semaforo

---

```
Data: S;                                     // Semaforo
1 wait(S);
2 Esecuzione della Sezione Critica;
3 signal(S);
```

---

Viene riportato a titolo di esempio un possibile pseudocodice per l’implementazione di un Semaforo.

---

**Algoritmo 3:** Pseudocodice di un Semaforo

---

```

1 Function Costruttore(int valoreIniziale):
2   | this. $Q_s$  = Nuova Coda di Descrittori di Processo vuota;
   | // Si richiede un valore iniziale da assegnare al
   |   Semaforo
3   | this. $S_v$  = valoreIniziale;
4 Function wait(Semaphore S):
   | // Verifica che il valore di  $S_v$  sia  $> 0$  e decrementalo,
   |   altrimenti inserisci il processo nella coda  $Q_s$ 
5   | if  $S.S_v > 0$  then
6   |   |  $S.S_v \leftarrow (S.S_v - 1)$ ;
7   | else
8   |   | Imposta lo stato del Processo su “Blocked” ed inserisci il suo
   |   |   descrittore in  $S.Q_s$ ;
9   | end if
10 Function signal(Semaphore S):
   | // Verifica che la coda  $Q_s$  non sia vuota e libera
   |   l’ultimo processo per poterne riprendere
   |   l’esecuzione, altrimenti incrementa il valore di  $S_v$ 
11  | if  $\exists$  Processo nella Coda  $S.Q_s$  then
12  |   | Rimuovi il descrittore del Processo da  $S.Q_s$  ed imposta il suo
   |   |   stato su “Ready”;
13  | else
14  |   |  $S.S_v \leftarrow (S.S_v + 1)$ ;
15  | end if

```

---

**Monitor**

Il Monitor[7] è anch’esso un ADT che tiene conto dei Vincoli di Sincronizzazione: implementa infatti da sé meccanismi di Mutua Esclusione per cui al suo interno potrà esserci al più uno e un solo Processo attivo. Una peculiarità nell’utilizzo del Monitor rispetto a quello di un più semplice Semaforo è quella di poter nascondere lo stato interno delle risorse condivise tra i vari processi, lasciando loro visibili unicamente le operazioni effettuabili su di esse.

In maniera più precisa un Monitor “è un costrutto che associa un insieme di

procedure ad una struttura dati comune a più Processi” [8]. Esso è composto dai seguenti elementi:

- Costruttore, che genera un’entità di tipo Monitor;
- Variabili Locali, le quali vanno a definire lo stato di un’istanza del tipo Monitor;
- Procedure, per l’interfacciamento con le risorse condivise dai Processi.

Un possibile pseudocodice per un Monitor potrebbe essere simile a quanto segue:

---

**Algoritmo 4:** Pseudocodice di un Monitor

---

```
// Variabili di Condizione
1 Condition variabileCondizione1;
2 Condition variabileCondizione2;
3 ...;
4 Condition variabileCondizioneN;
// Costruttore
5 Function Costruttore():
    | // Codice del Costruttore
// Procedure
6 Procedure Procedura1(Parametro1, Parametro2, ..., ParametroJ):
    | // Codice della Procedura 1
7 Procedure Procedura2(Parametro1, Parametro2, ..., ParametroK):
    | // Codice della Procedura 2
8 ...;
9 Procedure ProceduraN(Parametro1, Parametro2, ..., ParametroL):
    | // Codice della Procedura N
```

---

Un Monitor dispone di due livelli di Controllo per l’esecuzione esclusiva di operazioni sulle Risorse condivise:

**1. Primo Livello di Controllo:**

Le procedure di un Monitor sono eseguite in maniera Mutualmente Esclusiva, per cui qualora nel Monitor fosse presente un Processo in esecuzione ed un qualsiasi altro processo esterno volesse accedervi, quest’ultimo verrebbe posto in stato “*Blocked*” e si “addormenterebbe” all’esterno di esso.

## 2. Secondo Livello di Controllo:

È possibile per un Processo all'interno di un Monitor di "addormentarsi" su una Variabile di Condizione in quanto potrebbe non rispettare i requisiti imposti. In questo caso quest'ultimo passerà allo stato "*Blocked*" e posto in coda ad essa, così da lasciare spazio ad un altro Processo dall'esterno per poter eseguire.

Le variabili di Condizione di un Monitor dispongono di due primitive "*Wait*" e "*Signal*" che, seppur molto simili a quelle di un Semaforo, godono di leggere differenze: in questo caso infatti la "*Wait*" ha sempre un significato bloccante, mentre la "*Signal*" non produce effetti qualora non ci siano processi in coda.

## Scambio di Messaggi

Altro meccanismo di Sincronizzazione tra Processi Interagenti è lo scambio di Messaggi, utile in caso di interazioni di tipo Cooperativo. Per descriverne il funzionamento è importante definire alcune componenti quali le Primitive di Comunicazione "*Send*" e "*Receive*" ed il Canale di Comunicazione.

Vista la natura autoesplicativa delle Primitive, si pone ora enfasi sulla definizione del Canale di Comunicazione. Esso deve andare a determinare quali siano le coppie di Processi comunicanti e come identificarle (e quindi scegliere se utilizzare la Struttura Mailbox), se fruire o meno della "*Bufferizzazione*" e la lunghezza dei messaggi. Si noti che ciascuna di queste decisioni debba - almeno in linea teorica - essere ortogonale alle altre due.

Le scelte implementative sopracitate vengono analizzate a seguire:

### • Tecnica di Identificazione dei Processi Comunicanti

L'Identificazione dei Processi "*Sender*" e "*Receiver*" può avvenire sia direttamente che indirettamente.

Nel primo caso si va ad utilizzare l'identificativo dei processi come parametro delle primitive, come sotto riportato.

```
/*
 * R ed S sono rispettivamente gli Identificatori del Processo "Sender"
 *   ↪ e del Processo "Receiver".
 * La Variabile "message" sta ad indicare il messaggio inviato dal
 *   ↪ Processo "Sender" e che il Processo "Receiver" dovrà ricevere.
```

```
*/
send(R, message);
receive(S, message);
```

È anche possibile creare uno schema di tipo “Molti a Uno”, “Uno a Molti” o “Molti a Molti” tramite l’utilizzo di speciali Identificatori all’interno delle primitive “Send” e “Receive”.

```
// Molti a Uno
send(R, message); // Invia ad uno specifico Ricevitore
receive(ID, message); // Riceve da qualsiasi Mittente

// Uno a Molti
send(setOfReceivers, message); // Invia a piu' Ricevitori
receive(S, message); // Riceve dallo specifico Mittente "S"

// Molti a Molti
send(setOfReceivers, message); // Invia a piu' Ricevitori
receive(ID, message); // Riceve da qualsiasi Mittente
```

Per implementare un tipo di Identificazione Indiretta si va ad interporre un interlocutore terzo - la “*Mailbox*” - che si occupa di permettere ai Mittenti di inviare lei un messaggio, il quale verrà letto dai Destinatari a lei collegati.

- **Utilizzo della tecnica di “*Bufferizzazione*”**

L’utilizzo di tale funzionalità permette una comunicazione asincrona che consente un maggiore parallelismo all’applicativo. Il “*Buffer*” può essere di dimensione limitata o infinita. È possibile l’utilizzo di Messaggi di “*Acknowledgement*” - o ACK - da parte del Processo “*Receiver*” per confermare la ricezione del Messaggio. La corretta sequenza di ricezione potrebbe non essere garantita in caso di *Routing* Dinamico dei Messaggi, per cui si rende necessario l’utilizzo di una numerazione per l’ordinamento.

- **Scelta della lunghezza dei Messaggi**

È infine possibile scegliere se implementare dei Messaggi a lunghezza fissa, rendendo l’implementazione semplice ma spreco di memoria; o utilizzare Messaggi di lunghezza variabile, che risolvono il problema dei primi al costo di una più complessa realizzazione. Nei Sistemi Operativi moderni viene in linea generale adottato questo secondo approccio: ad esempio in UNIX vengono utilizzati messaggi di lunghezza

variabile con dimensione minima di 1 Byte.

### 1.5.3 Modelli Logici per la Concorrenza

Nel tempo sono stati definiti due Modelli Logici di riferimento per la gestione concorrenziale dei Processi: il Modello ad Ambiente Globale ed il Modello ad Ambiente Locale.

#### Modello ad Ambiente Globale

In tale contesto un'applicazione viene descritta come un insieme di Processi e Risorse, ove queste ultime sono una qualsiasi entità fisica o logica di cui uno o più Processi dell'applicazione stessa necessitano per portare a termine la loro esecuzione.

Data la natura condivisa di questo Ambiente sono possibili interazioni sia competitive che cooperative per l'utilizzo e lo scambio di risorse tra i Processi.

#### Modello ad Ambiente Locale

Definito successivamente, in questo Modello un'applicazione viene descritta come un insieme di Processi, ognuno dei quali va ad operare nel proprio ambiente utilizzando le proprie risorse, inaccessibili agli altri Processi della stessa applicazione.

La diversa concezione porta alla semplificazione delle interazioni tra i processi, rendendo possibili unicamente quelle di tipo cooperativo.

## 1.6 Processi Pesanti e Processi Leggeri

Un Sistema Operativo può supportare due diverse tipologie di Processi, le quali vengono di seguito descritte.

### 1.6.1 Processi Pesanti

I Processi Pesanti dispongono di un loro Program Counter, di un loro stato dei vari Registri della CPU, di un proprio *Stack*, del proprio Codice e dei

Propri Dati, per questo vengono identificati tramite la ennupla:

$$\textit{Processo_Pesante} = \{\textit{Program Counter}, \textit{Registri}, \textit{Stack}, \textit{Codice}, \textit{Dati}\}$$

Com'è facilmente intuibile Processi Pesanti distinti eseguono Codice distinto ed accedono a Dati distinti, non condividendo memoria ed ispirandosi quindi al Modello ad Ambiente Locale. La comunicazione tra più Processi pesanti avviene tramite l'utilizzo di tecniche quali lo scambio di Messaggi data la natura cooperativa delle loro interazioni e la condivisione di risorse è complessa, richiedendo una porzione di Memoria condivisa.

### 1.6.2 Processi Leggeri - “*Thread*”

I Processi Leggeri - o *Thread* - sono invece identificati dalla ennupla:

$$\textit{Thread} = \{\textit{Program Counter}, \textit{Registri}, \textit{Stack}\}$$

Più *Thread* correlati da un punto di vista logico condividono un'area di dati e codice tra loro formando un *Task* e ciascuno di essi rappresenta differenti contesti di esecuzione che risultano indipendenti all'interno dello stesso. Da ciò segue che i *Task* sono invece identificati da una ennupla del tipo:

$$\textit{Task} = \{\textit{Thread}_1, \textit{Thread}_2, \dots, \textit{Thread}_n, \textit{Codice}, \textit{Dati}\}$$

Alla luce di quanto illustrato è possibile immaginare un Processo Pesante come un *Task* composto da un solo *Thread*.

Si noti come sia comunque necessario, ai fini del *Context Switching*, mantenere un Descrittore di Processo (*Process Control Block* - *PCB*) per ogni Processo Leggero, rendendo più complesso il *PCB* del *Task* che va ad incorporare anche quelli dei suoi relativi *Thread*. L'implementazione di Processi Leggeri permette tuttavia operazioni di Cambio di Contesto all'interno dello stesso *Task* nettamente più veloci della controparte, in quanto i *PCB* dei singoli *Thread* risulta molto meno verboso di quello di un singolo Processo Pesante non contenendo alcuna informazione riguardo Codice e Dati.

In termini implementativi, i *Thread* di uno stesso *Task* condividono lo spazio di indirizzamento, le variabili globali e la tabella dei File aperti proprie del *Task*, ispirandosi quindi al Modello ad Ambiente Globale. Questa so-

miglianza implica l'insorgenza di problematiche quali le cosiddette “*Race Conditions*” date da interazioni di tipo sia Competitivo che Cooperativo: analogamente a quanto anticipato in precedenza, andranno di conseguenza implementate misure atte alla sincronizzazione tra i vari Processi Leggeri del *Task*.

### Modalità di Realizzazione

I *Thread* possono essere implementati su due distinti livelli:

#### 1. Livello Kernel

Il Sistema Operativo gestisce autonomamente l'avvicinarsi dei vari Processi Leggeri all'interno dell'Elaboratore, mettendo inoltre a disposizione i vari strumenti di sincronizzazione precedentemente analizzati.

#### 2. Livello Utente

In questo caso il Sistema Operativo vede unicamente l'intero *Task* come un comune Processo Pesante, quindi il Cambio di Contesto tra *Thread* di uno stesso *Task* avviene senza interrompere il Sistema Operativo. Questo vantaggio viene controbilanciato dalla necessità di bloccare l'intero *Task* alla chiamata di una *Routine* bloccante all'interno di un singolo *Thread*.

Ed il Sistema Operativo può supportare diversi Modelli Implementativi:

- **One-to-One**

Ogni *Thread* Utente viene mappato ad un solo *Thread Kernel*.

- **Many-to-One**

Più *Thread* Utente vengono mappati ad un solo *Thread Kernel*.

- **Many-to-Many**

Più *Thread* Utente vengono mappati a più *Thread Kernel*.



## Capitolo 2

# *Multi-Threading* in Java

Nel presente Capitolo verranno discusse le modalità di Programmazione permettano di utilizzare Processi Leggeri nel Linguaggio Java.

### 2.1 La Classe “Thread” e l’Interfaccia “Runnable”

Il Linguaggio di Programmazione Java mette a disposizione nella sua Libreria standard la classe “Thread” [9], la quale permette ad uno sviluppatore di scrivere Programmi che diano luce a Processi Leggeri. Per raggiungere tale obiettivo essa va ad implementare l’Interfaccia “Runnable” [10], la quale richiede la definizione del metodo “run()”.

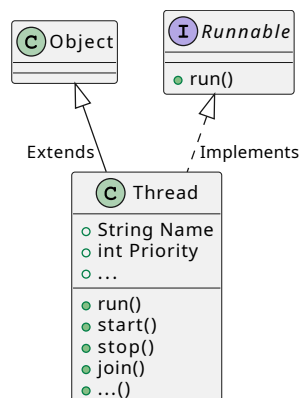


Figura 2.1: Ereditarietà della Classe **Thread** in Java. UML Generato tramite l’ausilio dell’editor PlantText[5].

Ogni oggetto di tipo *Thread* dispone di diversi Metodi e Attributi atti al-

la gestione del suo Flusso di Esecuzione e alla Sincronizzazione con altri Processi.

## 2.2 Stesura di un semplice Programma Multiprocesso

La stesura di un Programma che risulti nella generazione di più Processi Leggeri può avvenire in due distinte modalità, le quali vengono di seguito illustrate.

### 2.2.1 Implementazione come Sotto-Classe di Thread

Un primo approccio può essere quello di estendere la classe a cui si vuole donare la possibilità di essere eseguita in un Processo Leggero separato con la Classe `Thread`. Si riporta a seguire un esempio didattico per l'utilizzo dove il *Thread* principale andrà a creare un numero prestabilito di Processi Leggeri che dovranno stampare sullo Standard Output un messaggio per poi terminare l'esecuzione.

```
1  /*
2  * File: ClassExtendThread.java
3  * La classe mira a stampare sullo Standard Output l'identificativo del
   ↪ Thread assegnatogli dal Main. Estende la classe Thread.
4  */
5
6  public class ClassExtendThread extends Thread {
7
8      private int threadNumber;
9
10     public ClassExtendThread(int tNum) {
11         this.threadNumber = tNum;
12     }
13
14     @Override
15     public void run() {
16         System.out.println("Sono il Thread numero " + this.threadNumber + ".
   ↪ ");
17     }
18 }
```

## 2.2. STESURA DI UN SEMPLICE PROGRAMMA MULTIPROCESSO31

La classe dovrà sovrascrivere l'Implementazione di default del metodo “run()” fornita dalla classe **Thread**, inserendo il codice da eseguire all'avvio dell'esecuzione del Processo Leggero associato.

```
1  /*
2  * File: MainExtendThread.java
3  * La classe crea un numero NUM_OF_THREADS di Thread che estendano la classe
   ↪ Thread.
4  */
5
6  public class MainExtendThread {
7
8      private static final int NUM_OF_THREADS = 5;
9
10     public static void main(String[] args) {
11
12         System.out.println("Sono il Main e sto per creare i Thread.");
13
14         for(int i=0; i<NUM_OF_THREADS; i++) {
15             ClassExtendThread cet = new ClassExtendThread(i);
16             cet.start();
17         }
18
19         System.out.println("Sono il Main ed ho terminato l'esecuzione.");
20     }
21 }
```

La creazione di un oggetto di tipo “ClassExtendThread” munisce il Programmatore del metodo “start()” fornito dalla Classe **Thread** per eseguire il codice della Classe in un Processo Leggero separato.

### 2.2.2 Implementazione dell'Interfaccia Runnable

La controparte richiede un'implementazione molto simile, di seguito vengono riportate le differenze.

```
1  /*
2  * File: ClassImplementRunnable.java
3  * La classe mira a stampare sullo Standard Output l'identificativo del
   ↪ Thread assegnatogli dal Main. Implementa l'interfaccia Runnable.
4  */
5
6  public class ClassImplementRunnable implements Runnable {
7      ...
```

```
8 }

```

La Classe che prima andava ad estendere `Thread` ora deve implementare l'interfaccia `Runnable`, ma al di fuori di questo non sono necessarie modifiche nella sua struttura.

```

1  /*
2  * File: MainExtendThread.java
3  * La classe crea un numero NUM_OF_THREADS di Thread che estendano la classe
   ↪ Thread.
4  */
5  ...
6  for(int i=0; i<NUM_OF_THREADS; i++) {
7      ClassImplementRunnable cir = new ClassImplementRunnable(i);
8      Thread thread = new Thread(cir);
9      thread.start();
10 }
11 ...

```

La vera differenza risulta nell'utilizzo della Classe appena descritta: bisognerà infatti crearne un'istanza da passare al Costruttore di `Thread` in maniera tale da generare un oggetto di quest'ultima tipologia cosicché si possa utilizzare il metodo “`start()`” su di esso come illustrato nell'esempio precedente.

Notando che Java non permette di estendere più di una Classe alla volta, ma mantiene la possibilità di implementare più Interfacce contemporaneamente, si preferisce generalmente questo ultimo approccio nonostante necessiti di una implementazione più verbosa.

In ogni caso l'output dei Programmi sopracitati può assumere diverse forme a causa della mancanza di alcuna forma di sincronizzazione tra i vari Processi.

```

$ java MainImplementRunnable
Sono il Main e sto per creare i Thread.
Sono il Main ed ho terminato l'esecuzione.
Sono il Thread numero 2.
Sono il Thread numero 3.
Sono il Thread numero 0.
Sono il Thread numero 4.
Sono il Thread numero 1.

```

Si noti come l'esecuzione del *Thread* principale termini prima di quella degli altri Processi Leggeri da lui generati ma che questa circostanza non limiti in alcun modo questi ultimi, che continueranno la loro esecuzione fino alla rispettiva terminazione.

## 2.3 Alcuni Metodi per il Controllo del Flusso di Esecuzione

La classe **Thread** mette a disposizione alcuni metodi per la gestione del Flusso di Esecuzione dei singoli Processi Leggeri[11]. Ciò risulta utile nel caso in cui si voglia controllare il loro ordine di esecuzione o ancora la loro possibilità di eseguire.

### Il metodo **start()**

Il metodo “**start()**” permette di inizializzare l'esecuzione del Processo Leggero tramite l'invocazione del metodo “**run()**” poc'anzi citato.

### Il metodo **stop()**

Il metodo “**stop()**” permette di interrompere invece l'esecuzione del Processo Leggero attraverso l'utilizzo dell'eccezione “**ThreadDeath**”.

### Il metodo **suspend()**

Il metodo “**suspend()**” permette di sospendere momentaneamente l'esecuzione di un *Thread* fin tanto che su di lui non venga chiamato il metodo “**resume()**”, portandolo dallo stato “*Running*” allo stato “*Blocked*”.

### Il metodo **resume()**

Il metodo “**resume()**” permette di riprendere l'esecuzione di un *Thread* bloccato dal metodo “**suspend()**”, portandolo dallo stato “*Blocked*” allo stato “*Ready*”.

### Il metodo **sleep(int millis)**

Il metodo “**sleep()**” permette di portare il Processo Leggero nello stato “*Blocked*” (o “*Sleeping*” se si fa riferimento a sistemi \*NIX-Like) per un

determinato numero di millisecondi senza utilizzare cicli di Clock della CPU. Tale metodo solleva un'eccezione di tipo “`InterruptedException`” nel caso in cui tale Processo Leggero venisse interrotto da un altro Processo mentre è già in stato “*Sleeping*”, perciò il metodo viene incapsulato in un costrutto di tipo “*Try-Catch*”.

### Il metodo `destroy()`

Il metodo “`destroy()`” permette di eliminare il *Thread* senza effettuare alcuna pulizia, lasciando quindi allocati i vari Monitor da esso utilizzati e facilitando l'insorgere di una situazione di Blocco Critico.

### Il metodo `yield()`

Il metodo “`yield()`” permette al *Thread* di lasciare volontariamente la CPU ad un altro processo, implicando un *Re-Scheduling*.

### Il metodo `join()`

Il metodo “`join()`” consente al Processo Padre di attendere la terminazione del Processo Figlio prima di continuare con la propria esecuzione, garantendo un semplice meccanismo di sincronizzazione tra i due.

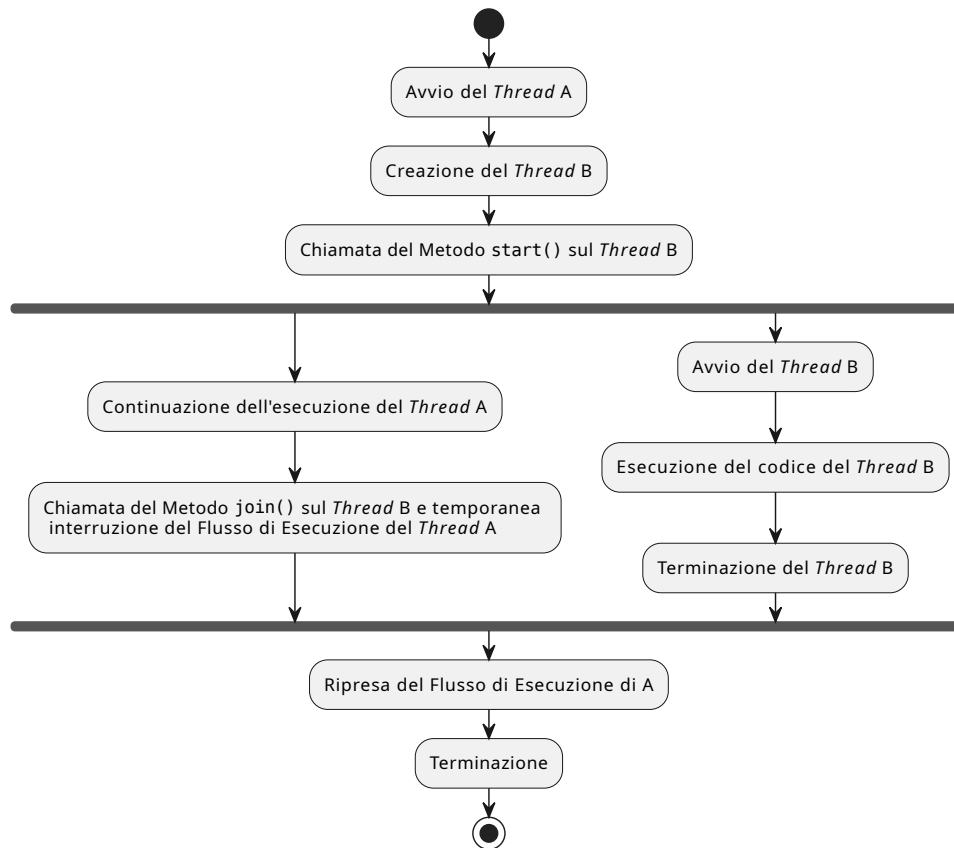


Figura 2.2: Flusso di Esecuzione di un Processo Padre ed un Processo Figlio nel caso di utilizzo del Metodo “`join()`”. Grafo Generato tramite l’ausilio dell’editor PlantText[5].

### 2.3.1 Problemi del Controllo del Flusso di Esecuzione

Così come discusso nel Capitolo 1 l’arbitraria interruzione dell’esecuzione di un Processo potrebbe portare a situazioni indesiderate: una o più risorse potrebbero risultare in uno stato non consistente, oppure ancora una risorsa che era stata acquisita dal *Thread* che ha subito l’interruzione avrebbe la possibilità rimanere bloccata, portando sotto le giuste ipotesi in uno stato di *Deadlock*. Per questi motivi si preferisce non utilizzare in maniera imperativa i Metodi “`stop()`”, “`suspend()`”, “`resume()`” o “`destroy()`”, i quali sono stati infatti deprecati.

### 2.3.2 Considerazioni sul Controllo del Flusso di Esecuzione

Alle primitive sopracitate si preferisce usufruire della possibilità di segnalare ai Processi la possibilità di sospendersi o terminare la loro esecuzione, in maniera tale da permettere loro di eseguire le proprie Sezioni Critiche e lasciare il Sistema in uno stato consistente. Un esempio di Metodo che permette al Processo Leggero di lasciare volontariamente la CPU è “`yield()`”. La natura delle moderne versioni della *Java Virtual Machine* implica poi l'utilizzo dello *Scheduler* del Sistema Operativo, il quale dovrà scegliere il prossimo Processo cui lasciare il controllo. Notando che tutti i moderni Algoritmi di *Scheduling* supportano l'utilizzo di un valore di Priorità per la scelta del Processo dalla Coda “*Ready*”, anche Java mette a disposizione un valore di Priorità per i propri Processi Leggeri: esso varia tra il valore minimo “`MIN_PRIORITY`” (0) ed il valore massimo “`MAX_PRIORITY`” (10) e viene impostato in maniera automatica - al netto di ulteriori istruzioni - su “`NORM_PRIORITY`” (5), risultando visualizzabile e personalizzabile rispettivamente tramite le Primitive “`getPriority()`” e “`setPriority()`”.

## Capitolo 3

# Sincronizzazione di Processi Leggeri in Java

Come discusso in precedenza l'utilizzo concorrenziale di risorse condivise da parte di due o più Processi può portare ad inconsistenze, per cui è buona norma utilizzare oltre alle Tecniche di Controllo del Flusso di Esecuzione illustrate poc'anzi, anche Tecniche di Sincronizzazione tra i Processi in questione in maniera tale da garantire una corretta esecuzione del Programma ed una corretta gestione delle suddette Risorse comuni.

### 3.1 Il Costrutto *Synchronized*

Java mette a disposizione del Programmatore un costrutto *ad hoc* chiamato “*Synchronized*”, il quale va a simulare l'utilizzo di un Monitor simile a quello definito in Concurrent Pascal illustrato in precedenza per la risorsa che viene protetta. Quest'ultima può essere un metodo o un blocco di codice, indicando quindi una Sezione Critica.

```
public synchronized void
    ↪ methodName() {
    // Metodo Protetto
}
```

(a) Esempio di dichiarazione di un Metodo *Synchronized*.

```
synchronized(Object) {
    // Blocco di Codice Protetto
}
```

(b) Esempio di dichiarazione di un Blocco di Codice *Synchronized*.

Figura 3.1: Esempio di utilizzo del Costrutto *Synchronized*.

Nel caso della Figura 3.1a si parla di Sincronizzazione Implicita, ed un solo *Thread* alla volta potrà eseguire il metodo protetto. Nel caso della Figura 3.1b si parla invece di Sincronizzazione Esplicita ed ancora una volta in ogni momento un solo Processo Leggero potrà eseguire tale blocco protetto. All'interno di un Metodo *Synchronized* è poi possibile utilizzare due speciali primitive:

- *wait()*

Il funzionamento è analogo all'omonima primitiva per un Monitor in Concurrent Pascal: permette infatti di sospendere il Processo chiamante su una coda associata all'oggetto dotato del suddetto metodo *Synchronized*, rilasciando quindi il Mutex ad esso connesso. Essendo analogo anche nel funzionamento al costrutto Monitor del Concurrent Pascal, una chiamata del Metodo *wait()* in un momento in cui nessun altro processo è sospeso viene persa, non avendo alcun effetto.

- *notify()*

Questo ha invece un comportamento assimilabile a quello della primitiva *signal()* sempre disponibile nel contesto di un Monitor in Concurrent Pascal e permette quindi di risvegliare il primo dei *Thread* sospesi sulla coda dell'Oggetto sul quale è stato invocato il metodo.

Ne esiste una variante “*notifyAll()*” che risveglia tutti i *Thread* “addormentati” sulla coda dell'Oggetto. Si noti che comunque un solo processo alla volta potrà riacquisire il *Lock* vista la sua natura mutualmente esclusiva.

## 3.2 Il costrutto *Lock*

# Bibliografia

- [1] Wikipedia. Bubble Sort, Giugno 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Bubble\\_sort](https://it.wikipedia.org/wiki/Bubble_sort). Pagina visitata il 27 dicembre 2023.
- [2] Wikipedia. Radix Sort, Maggio 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Radix\\_sort](https://it.wikipedia.org/wiki/Radix_sort). Pagina visitata il 27 dicembre 2023.
- [3] Wikipedia. Algoritmo di Dijkstra, Aprile 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Algoritmo\\_di\\_Dijkstra](https://it.wikipedia.org/wiki/Algoritmo_di_Dijkstra). Pagina visitata il 27 dicembre 2023.
- [4] Wikipedia. Algoritmo di Bellman-Ford, Maggio 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Algoritmo\\_di\\_Bellman-Ford](https://it.wikipedia.org/wiki/Algoritmo_di_Bellman-Ford). Pagina visitata il 27 dicembre 2023.
- [5] PlantUML, Graphviz, Ace Editor, Johan Sundström, Steven Nichols e Arwen Vaughan. PlantText UML Editor. Editor disponibile all'url: <https://www.planttext.com/>. Pagina visitata il 27 dicembre 2023.
- [6] Wikipedia. Semaforo, Agosto 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Semaforo\\_\(informatica\)](https://it.wikipedia.org/wiki/Semaforo_(informatica)). Pagina visitata il 27 dicembre 2023.
- [7] Wikipedia. Monitor, Marzo 2022. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Monitor\\_\(sincronizzazione\)](https://it.wikipedia.org/wiki/Monitor_(sincronizzazione)). Pagina visitata il 27 dicembre 2023.
- [8] Prof. Letizia Leonardi. Costrutti di Sincronizzazione, Ottobre 2022. Dispensa disponibile all'url: <https://www.didattica.agentgroup.unimo.it/didattica/ProgettazioneS0/Lucidi/CostruttiDISincronizzazione-3bis.pdf>. Documento Protetto da

Password. Pagina visitata il 27 dicembre 2023.

- [9] Oracle Corp. *Class Thread*. Materiale disponibile all'url: <https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html>. Pagina visitata il 27 dicembre 2023.
- [10] Oracle Corp. *Interface Runnable*. Materiale disponibile all'url: <https://docs.oracle.com/javase/8/docs/api/java/lang/Runnable.html>. Pagina visitata il 27 dicembre 2023.
- [11] Prof. Letizia Leonardi, PhD Mariachiara Puviani. Programmazione Concorrente in Java, Ottobre 2014. Dispensa disponibile all'url: <https://didattica.agentgroup.unimore.it/wiki/images/5/5e/SeminarioJava2014.pdf>. Pagina visitata il 27 dicembre 2023.